

First Class In C++

Adapted from : <http://www.cplusplus.com/doc/tutorial/tut1-1.html>

Structure of a C++ program

Probably the best way to start learning a programming language is with a program. So here is our first program:

```
// my first program in C++
```

```
#include <iostream.h>
```

```
int main ()
```

```
{  
    cout << "Hello World!";  
    return 0;  
}
```

Hello World!

The left side shows the source code for our first program, which we can name, for example, hiworld.cpp. The right side shows the result of the program once compiled and executed. The way to edit and compile a program depends on the compiler you are using. Depending on whether it has a Development Interface or not and on its version. Consult section [compilers](#) and the manual or help included with your compiler if you have doubts on how to compile a C++ console program.

The previous program is the first program that most programming apprentices write, and its result is the printing on screen of the "Hello World!" sentence. It is one of the simpler programs that can be written in C++, but it already includes the basic components that every C++ program has. We are going to take a look at them one by one:

// my first program in C++

This is a comment line. All the lines beginning with two slash signs (//) are considered comments and do not have any effect on the behavior of the program. They can be used by the programmer to include short explanations or observations within the source itself. In this case, the line is a brief description of what our program does.

#include <iostream.h>

Sentences that begin with a pound sign (#) are directives for the preprocessor. They are not executable code lines but indications for the compiler. In this case the sentence **#include <iostream.h>** tells the compiler's preprocessor to include the **iostream** standard header file. This specific file includes the declarations of the basic standard input-output library in C++, and it is included because its functionality is used later in the program.

int main ()

This line corresponds to the beginning of the **main** function declaration. The **main** function is the point by where all C++ programs begin their execution. It is independent of whether it is at the beginning, at the end or in the middle of the code - its content is always the first to be executed when a program starts. In addition, for that same reason, it is essential that all C++ programs have a **main** function.

main is followed by a pair of parenthesis **()** because it is a function. In C++ all functions are followed by a pair of parenthesis **()** that, optionally, can include arguments within them. The

content of the **main** function immediately follows its formal declaration and it is enclosed between curly brackets (**{}**), as in our example.

cout << "Hello World";

This instruction does the most important thing in this program. **cout** is the standard output stream in C++ (usually the screen), and the full sentence inserts a sequence of characters (in this case "Hello World") into this output stream (the screen). **cout** is declared in the `iostream.h` header file, so in order to be able to use it that file must be included.

Notice that the sentence ends with a semicolon character (**;**). This character signifies the end of the instruction and must be included after every instruction in any C++ program (one of the most common errors of C++ programmers is indeed to forget to include a semicolon **;** at the end of each instruction).

return 0;

The **return** instruction causes the **main()** function finish and return the code that the instruction is followed by, in this case **0**. This is the most usual way to terminate a program that has not found any errors during its execution. As you will see in coming examples, all C++ programs end with a sentence similar to this.

Therefore, you may have noticed that not all the lines of this program did an action. There were lines containing only comments (those beginning by `//`), lines with instructions for the compiler's preprocessor (those beginning by `#`), then there were lines that initiated the declaration of a function (in this case, the **main** function) and, finally lines with instructions (like the call to **cout <<**), these last ones were all included within the block delimited by the curly brackets (**{}**) of the **main** function.

The program has been structured in different lines in order to be more readable, but it is not compulsory to do so. For example, instead of

```
int main ()
{
    cout << " Hello World ";
    return 0;
}
```

we could have written:

```
int main () { cout << " Hello World "; return 0; }
```

in just one line and this would have had exactly the same meaning.

In C++ the separation between instructions is specified with an ending semicolon (**;**) after each one. The division of code in different lines serves only to make it more legible and schematic for humans that may read it.

Here is a program with some more instructions:

```
// my second program in C++
```

```
#include <iostream.h>
```

```
int main ()
{
```

Hello World! I'm a C++ program

```
cout << "Hello World! ";
cout << "I'm a C++ program";
return 0;
}
```

In this case we used the **cout** << method twice in two different instructions. Once again, the separation in different lines of the code has just been done to give greater readability to the program, since **main** could have been perfectly defined thus:

```
int main () { cout << " Hello World! "; cout << " I'm to C++ program "; return 0; }
We were also free to divide the code into more lines if we considered it convenient:
int main ()
{
    cout <<
        "Hello World!";
    cout
        << "I'm a C++ program";
    return 0;
}
```

And the result would have been exactly the same than in the previous examples.

Preprocessor directives (those that begin by #) are out of this rule since they are not true instructions. They are lines read and discarded by the preprocessor and do not produce any code. These must be specified in their own line and do not require the include a semicolon (;) at the end.

Comments.

Comments are pieces of source code discarded from the code by the compiler. They do nothing. Their purpose is only to allow the programmer to insert notes or descriptions embedded within the source code.

C++ supports two ways to insert comments:

```
// line comment
/* block comment */
```

The first of them, the line comment, discards everything from where the pair of slash signs (//) is found up to the end of that same line. The second one, the block comment, discards everything between the /* characters and the next appearance of the */ characters, with the possibility of including several lines.

We are going to add comments to our second program:

```
/* my second program in C++
   with more comments */

#include <iostream.h>

int main ()
{
```

Hello World! I'm a C++ program

```
cout << "Hello World! "; // says Hello
World!
cout << "I'm a C++ program"; // says I'm a
C++ program
return 0;
}
```

If you include comments within the sourcecode of your programs without using the comment characters combinations `//`, `/*` or `*/`, the compiler will take them as if they were C++ instructions and, most likely causing one or several error messages

Variables. Data types. Constants

The usefulness of the "Hello World" programs shown in the previous section are something more than questionable. We had to write several lines of code, compile them, and then execute the resulting program just to obtain a sentence on the screen as result. It is true that it would have been much faster to simply write the output sentence by ourselves, but programming is not limited only to printing texts on screen. In order to go a little further on and to become able to write programs that perform useful tasks that really save us work we need to introduce the concept of the **variable**.

Let's think that I ask you to retain the number 5 in your mental memory, and then I ask you to also memorize the number 2. You have just stored two values in your memory. Now, if I ask you to add 1 to the first number I said, you should be retaining the numbers 6 (that is 5+1) and 2 in your memory. Values that we could now subtract and obtain 4 as the result.

All this process that you have made is a simile of what a computer can do with two variables. This same process can be expressed in C++ with the following instruction set:

```
a = 5;
b = 2;
a = a + 1;
result = a - b;
```

Obviously this is a very simple example since we have only used two small integer values, but consider that your computer can store millions of numbers like these at the same time and conduct sophisticated mathematical operations with them.

Therefore, we can define a variable as a portion of memory to store a determined value.

Each variable needs an identifier that distinguishes it from the others, for example, in the previous code the variable identifiers were **a**, **b** and **result**, but we could have called the variables any names we wanted to invent, as long as they were valid identifiers.

Identifiers

A valid identifier is a sequence of one or more letters, digits or underline symbols (`_`). The length of an identifier is not limited, although for some compilers only the 32 first characters of an identifier are significant (the rest are not considered).

Neither spaces nor marked letters can be part of an identifier. Only letters, digits and underline characters are valid. In addition, variable identifiers should always begin with a letter. They can also begin with an underline character (_), but this is usually reserved for external links. In no case they can begin with a digit.

Another rule that you have to consider when inventing your own *identifiers* is that they cannot match any **key word** of the C++ language nor your compiler's specific ones since they could be confused with these. For example, the following expressions are always considered key words according to the ANSI-C++ standard and therefore they must not be used as identifiers:

asm, auto, bool, break, case, catch, char, class, const, const_cast, continue, default, delete, do, double, dynamic_cast, else, enum, explicit, extern, false, float, for, friend, goto, if, inline, int, long, mutable, namespace, new, operator, private, protected, public, register, reinterpret_cast, return, short, signed, sizeof, static, static_cast, struct, switch, template, this, throw, true, try, typedef, typeid, typename, union, unsigned, using, virtual, void, volatile, wchar_t

Additionally, alternative representations for some operators do not have to be used as identifiers since they are reserved words under some circumstances:

and, and_eq, bitand, bitor, compl, not, not_eq, or, or_eq, xor, xor_eq

Your compiler may also include some more specific reserved keywords. For example, many compilers which generate 16 bit code (like some compilers for DOS) also include **far, huge** and **near** as key words.

Very important: The C++ language is "case sensitive", that means that an identifier written in capital letters is not equivalent to another one with the same name but written in small letters. Thus, for example the variable **RESULT** is not the same as the variable **result** nor the variable **Result**.

Data types

When programming, we store the variables in our computer's memory, but the computer must know what we want to store in them since storing a simple number, a letter or a large number is not going to occupy the same space in memory.

Our computer's memory is organized in bytes. A byte is the minimum amount of memory that we can manage. A byte can store a relatively small amount of data, usually an integer between 0 and 255 or one single character. But in addition, the computer can manipulate more complex data types that come from grouping several *bytes*, such as long numbers or numbers with decimals. Next you have a list of the existing fundamental data types in C++, as well as the range of values that can be represented with each one of them:

DATA TYPES

Name	Bytes*	Description	Range*
char	1	character or integer 8 bits length.	signed: -128 to 127 unsigned: 0 to 255
short	2	integer 16 bits length.	signed: -32768 to 32767 unsigned: 0 to 65535
long	4	integer 32 bits length.	signed: -2147483648 to 2147483647 unsigned: 0 to 4294967295
int	*	Integer. Its length traditionally depends	See short, long

		on the length of the system's Word type , thus in MSDOS it is 16 bits long, whereas in 32 bit systems (like Windows 9x/2000/NT and systems that work under protected mode in x86 systems) it is 32 bits long (4 bytes).	
float	4	floating point number.	3.4e + / - 38 (7 digits)
double	8	double precision floating point number.	1.7e + / - 308 (15 digits)
long double	10	long double precision floating point number.	1.2e + / - 4932 (19 digits)
bool	1	Boolean value. It can take one of two values: true or false NOTE: this is a type recently added by the ANSI-C++ standard. Not all compilers support it. Consult section <u>bool type</u> for compatibility information.	true or false
wchar_t	2	Wide character. It is designed as a type to store international characters of a two-byte character set. NOTE: this is a type recently added by the ANSI-C++ standard. Not all compilers support it.	wide characters

* Values of columns Bytes and Range may vary depending on your system. The values included here are the most commonly accepted and used by almost all compilers.

In addition to these fundamental data types there also exist the pointers and the **void** parameter type specification, that we will see later.

Declaration of variables

USING VARIABLES IN PROGRAMMING

A variable is like a box in the computer's memory. It is used to hold a piece of data, for instance, a number or a string. A variable has a name, so that it can be referred to, like a box with a label on it.

A variable has a piece of data placed in it using an equals sign:

$$x = 2.6$$

However, C++ does require you to set the variable up before you use it. You have to list the variables that you are going to use. This is normally done after the `main () {` statement, although you will find they can be set up in other places as well. This process is called **declaring variables**.

Variables are declared with the `var` command. You have to specify the name of the variable and also what **data type** it is.

In order to use a variable in C++, we must first declare it specifying which of the data types above we want it to be. The syntax to declare a new variable is to write the data type specifier that we want (like **int**, **short**, **float**...) followed by a valid variable identifier. For example:

```
int a;  
float mynumber;
```

Are valid declarations of variables. The first one declares a variable of type **int** with the identifier **a**. The second one declares a variable of type **float** with the identifier **mynumber**. Once declared, variables **a** and **mynumber** can be used within the rest of their scope in the program.

If you need to declare several variables of the same type and you want to save some writing work you can declare all of them in the same line separating the identifiers with commas. For example:

```
int a, b, c;  
declares three variables (a, b and c) of type int , and has exactly the same meaning as if we had  
written:  
int a;  
int b;  
int c;
```

Integer data types (**char**, **short**, **long** and **int**) can be signed or unsigned according to the range of numbers that we need to represent. Thus to specify an integer data type we do it by putting the keyword **signed** or **unsigned** before the data type itself. For example:

```
unsigned short NumberOfSons;  
signed int MyAccountBalance;  
By default, if we do not specify signed or unsigned it will be assumed that the type is signed,  
therefore in the second declaration we could have written:  
int MyAccountBalance;  
with exactly the same meaning and since this is the most usual way, few source codes include the  
keyword signed as part of a compound type name.
```

The only exception to this rule is the **char** type that exists by itself and it is considered a different type than **signed char** and **unsigned char**.

Finally, **signed** and **unsigned** may also be used as a simple types, meaning the same as **signed int** and **unsigned int** respectively. The following two declarations are equivalent:

```
unsigned MyBirthYear;  
unsigned int MyBirthYear;
```

To see what variable declaration looks like in action in a program, we are going to show the C++ code of the example about your mental memory proposed at the beginning of this section:

```
// operating with variables  
#include <iostream.h>  
  
int main ()  
{  
    // declaring variables:  
    int a, b;  
    int result;  
  
    // process:
```

4

```
a = 5;
b = 2;
a = a + 1;
result = a - b;
// print out the result:
cout << result;

// terminate the program:
return 0;
}
```

Do not worry if something about the variable declarations looks a bit strange to you. You will see the rest in detail in coming sections.

Initialization of variables

When declaring a local variable, its value is undetermined by default. But you may want a variable to store a concrete value the moment that it is declared. In order to do that, you have to append an equal sign followed by the value wanted to the variable declaration:

type identifier = initial_value ;

For example, if we want to declare an **int** variable called **a** that contains the value **0** at the moment in which it is declared, we could write:

```
int a = 0;
```

Additionally to this way of initializing variables (known as c-like), C++ has added a new way to initialize a variable: by enclosing the initial value between parenthesis **()**:

type identifier (initial_value) ;

For example:

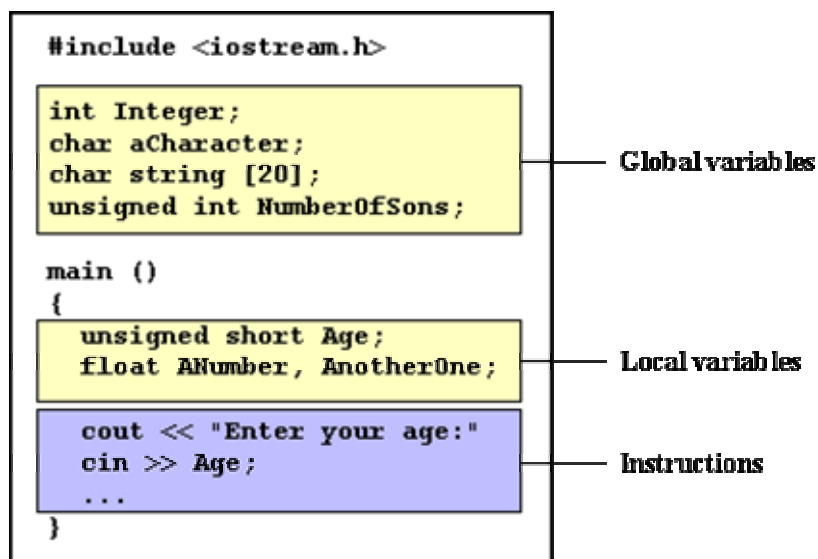
```
int a (0);
```

Both ways are valid and equivalent in C++.

Scope of variables

All the variables that we are going to use must have been previously declared. An important difference between the C and C++ languages, is that in C++ we can declare variables anywhere in the source code, even between two executable sentences, and not only at the beginning of a block of instructions, like happens in C.

Anyway, it is recommended under some circumstances to follow the indications of the C language when declaring variables, since it can be useful when debugging a program to have all the declarations grouped together. Therefore, the traditional C-like way to declare variables is to include their declaration at the beginning of each function (for local variables) or directly in the body of the program outside any function (for global variables).



Global variables can be referred to anywhere in the code, within any function, whenever it is after its declaration.

The scope of the **local variables** is limited to the code level in which they are declared. If they are declared at the beginning of a function (like in **main**) their scope is the whole **main** function. In the example above, this means that if another function existed in addition to `main()`, the local variables declared in **main** could not be used in the other function and vice versa.

In C++, the scope of a local variable is given by the block in which it is declared (a block is a group of instructions grouped together within curly brackets `{}` signs). If it is declared within a function it will be a variable with function scope, if it is declared in a loop its scope will be only the loop, etc...

In addition to **local** and **global** scopes there exists external scope, that causes a variable to be visible not only in the same source file but in all other files that will be linked together.

Constants: Literals.

A constant is any expression that has a fixed value. They can be divided in Integer Numbers, Floating-Point Numbers, Characters and Strings.

Integer Numbers

1776
707
-273

they are numerical constants that identify integer decimal numbers. Notice that to express a numerical constant we do not need to write quotes (") nor any special character. There is no doubt that it is a constant: whenever we write **1776** in a program we will be referring to the value 1776.

In addition to decimal numbers (those that all of us already know) C++ allows the use as literal constants of octal numbers (base 8) and hexadecimal numbers (base 16). If we want to express an octal number we must precede it with a **0** character (zero character). And to express a hexadecimal number we have to precede it with the characters **0x** (zero, x). For example, the following literal constants are all equivalent to each other:

75 // decimal
0113 // octal
0x4b // hexadecimal

All of them represent the same number: 75 (seventy five) expressed as a radix-10 number, octal and hexadecimal, respectively.

[Note: You can find more information on hexadecimal and octal representations in the document [Numerical radixes](#)]

Floating Point Numbers

They express numbers with decimals and/or exponents. They can include a decimal point, an **e** character (that expresses "by ten at the Xth height", where X is the following integer value) or both.

```
3.14159 // 3.14159
6.02e23 // 6.02 x 1023
1.6e-19 // 1.6 x 10-19
3.0     // 3.0
```

these are four valid numbers with decimals expressed in C++. The first number is PI, the second one is the number of Avogadro, the third is the electric charge of an electron (an extremely small number) -all of them approximated- and the last one is the number 3 expressed as a floating point numeric literal.

Characters and strings

There also exist non-numerical constants, like:

```
'z'
'p'
"Hello world"
"How do you do?"
```

The first two expressions represent single characters, and the following two represent strings of several characters. Notice that to represent a single character we enclose it between single quotes (') and to express a string of more than one character we enclose them between double quotes (").

When writing both single characters and strings of characters in a constant way, it is necessary to put the quotation marks to distinguish them from possible variable identifiers or reserved words. Notice this:

```
x
'x'
x refers to variable x, whereas 'x' refers to the character constant 'x'.
```

Character constants and string constants have certain peculiarities, like the **escape codes**. These are special characters that cannot be expressed otherwise in the sourcecode of a program, like *newline* (\n) or *tab* (\t). All of them are preceded by an inverted slash (\). Here you have a list of such escape codes:

\n	newline
\r	carriage return
\t	tabulation
\v	vertical tabulation
\b	backspace
\f	page feed
\a	alert (beep)
\'	single quotes (')

\"	double quotes (")
\?	question (?)
\\	inverted slash (\)

For example:

```
'\n'
```

```
'\t'
```

```
"Left \t Right"
```

```
"one\ntwo\nthree"
```

Additionally, you can express any character by its numerical ASCII code by writing an inverted slash bar character (\) followed by the ASCII code expressed as an octal (radix-8) or hexadecimal (radix-16) number. In the first case (octal) the number must immediately follow the inverted slash (for example \23 or \40), in the second case (hexadecimal), you must put an x character before the number (for example \x20 or \x4A).

[Consult the document [ASCII Code](#) for more information about this type of escape code].

Constants of string of characters can be extended by more than a single code line if each code line ends with an inverted slash (\):

```
"string expressed in \
two lines"
```

You can also concatenate several string constants separating them by one or several blankspaces, tabulators, newline or any other valid blank character:

```
"we form" "a single" "string" "of characters"
```

Defined constants (**#define**)

You can define your own names for constants that you use quite often without having to resort to variables, simply by using the **#define** preprocessor directive. This is its format:

```
#define identifier value
```

For example:

```
#define PI 3.14159265
```

```
#define NEWLINE '\n'
```

```
#define WIDTH 100
```

they define three new constants. Once they are declared, you are able to use them in the rest of the code as any if they were any other constant, for example:

```
circle = 2 * PI * r;
```

```
cout << NEWLINE;
```

In fact the only thing that the compiler does when it finds **#define** directives is to replace literally any occurrence of the them (in the previous example, **PI**, **NEWLINE** or **WIDTH**) by the code to which they have been defined (**3.14159265**, **'\n'** and **100**, respectively). For this reason, **#define** constants are considered *macro constants*.

The **#define** directive is not a code instruction, it is a directive for the preprocessor, therefore it assumes the whole line as the directive and does not require a semicolon (;) at the end of it. If you include a semicolon character (;) at the end, it will also be added when the preprocessor will substitute any occurrence of the defined constant within the body of the program.

declared constants (const)

With the **const** prefix you can declare constants with a specific type exactly as you would do with a variable:

```
const int width = 100;
```

```
const char tab = '\t';
```

```
const zip = 12440;
```

In case that the type was not specified (as in the last example) the compiler assumes that it is type **int**

```
// Your first program with variables!
```

```
#include <iostream.h>
```

```
void main ()
```

```
{
```

```
    int x;
```

```
    float my_variable;
```

```
    x = 10;
```

```
    my_variable = -262.344;
```

```
    cout << "The value of x is " << x << endl;
```

```
    cout << "The value of my_variable is "
```

```
        << my_variable << endl;
```

```
}
```

Variable Names

There are two variables in this program, `x` and `my_variable`. You can see that variables don't have to have names which are only one letter. Indeed, they can have as many letters as you like in the name, with only two restrictions:

- The name can contain upper or lower case letters, underscores (`_`) or digits,
- The first character of the variable name must be a letter or an underscore.

Here are some legal variable names: `my_variable`; `age`; `VAR1`; `Variable_123`; `This_Is_My_Name`; `_age`

Here are some illegal variable names, with the reason why they aren't acceptable:

`Var 2` Contains a space in the name

`3Name` The name starts with a number

`Also.a,variable&name?` The name contains illegal characters

Assigning variables to values

A variable has a name. It also has a *value*. You can set a variable to a value using the equals sign (`=`). The crucial lines in the program above are

```
x = 10;
```

```
my_variable = -262.344;
```

This sets `x` to the value 10 and `my_variable` to the value -262.344. Not every variable can be set to any value. You restrict the values of variables when you decide on their data type.

Initialising variables during declaration

Variables can be set to initial values at the point where they are declared. It is done by putting the = 2; (or whatever the number is) after the int x part.

```
int my_variable = 1000;
```

```
float pi = 3.141592;
```

```
char initial_letter = 'm';
```

This sets up the three variables to the values **1000**, **3.141592** and **m**. Note that the character **m** has single quotation marks round it, not double quotation marks. If you put double quotation marks round, the compiler reports an error.

Data Types

You will see in the example program the lines

```
int x;  
float my_variable;
```

This tells C++ that the variable x is going to be an integer. This means a whole number, which can be positive (any value up to +32767), negative (down to -32768) or zero. Having declared x using int, the following statements would be legal:

```
x = -6610;  
x = 234;  
x = +2;  
x = 0;
```

and these statements would be illegal:

```
x = 1.2;    Not a whole number  
x = 10.0;   Although technically a whole number, the decimal point  
            convinces C++ that 10.0 is a decimal.  
x = 900000; Out of the valid range for int
```

Having said that, I should point out that C++ is very tolerant of values given to variables - tolerant to a fault, in fact. If you tried to set an **int** variable to a decimal number, C++ would not complain.

However, it might not produce the number that you expect. Take a look at this sample program:

```
#include <iostream.h>
```

```
void main ()
```

```
{ int whole_num;  
  whole_num = 0.5667;
```

```
  char single_letter;  
  single_letter = 65;
```

```
  cout << "The value of whole_num is "  
    << whole_num << endl;  
  cout << "The value of single_letter is "  
    << single_letter << endl;
```

```
}
```

However, you will get some rather strange results. If you try the program above, you will get this on the screen:

The value of whole_num is 0
The value of single_letter is A

So where did that come from? Well, C++ tried its best, and set **whole_num** to a whole number. It rounds 0.5667 down (always rounds down!) to 0. It also turns 65 into a character, which comes out as letter A, and that becomes the value of **single_letter**. There is a list of the characters that the programming language understands (called the American Standard Code for Information Interchange - ASCII for short), and A is character number 65 in that list.

The sure way of not getting spurious results for your variables is to make sure that you only set them to the types for which they are intended. Anyone who writes computer code like the program above *deserves everything that happens to them!!*

Initializing variables during declaration

Variables can be set to initial values at the point where they are declared. It is done by putting the = 2; (or whatever the number is) after the int x part.

```
int my_variable = 1000;  
float pi = 3.141592;  
char initial_letter = 'm';
```

This sets up the three variables to the values **1000**, **3.141592** and **m**. Note that the character **m** has single quotation marks round it, not double quotation marks. If you put double quotation marks round, the compiler reports an error.

Variable Arithmetic

You can produce arithmetic expressions with variables in them just as you can with numbers. The same rules apply, of course - use * for multiply and / for divide.

```
// This statement should write 15 on the screen  
int a,b,c,d;  
a = 4;  
b = 3;  
c = 7;  
d = 10;  
cout << a + b * c - 10;
```

Variables can be assigned values using arithmetic just as they can for simple numbers:

```
x = 2 * y - (6 + total_value * 10) / 8;
```

The division operator has a slight quirk when used with integers. Look at this:

```
// The result of this program is unexpected!  
#include <iostream.h>  
void main ()  
{ int x;  
  x = 13 / 6;  
  cout << x  
}
```

The result of 13 / 6 is a decimal, and you would expect the compiler to complain when you try to set it to an integer. However, the compiler realises that x is an integer, and simply loses the decimal part of the answer. The program produces the number 2 and forgets the remainder.

If you want the remainder of a division sum, use the operator %. This gives just the remainder (called the modulo value) and forgets the normal part of the division:

```
// Modulo arithmetic
#include <iostream.h>
void main ()
{ int x;
  x = 13 % 6;
  cout << x
}
```

This program would produce the value 1, which is the remainder when 13 is divided by 6.

So far you have been doing arithmetic and printing the results on the screen. The only way in which you have been able to change the numbers is to alter the program and then recompile it. It would be nice to alter the numbers while the program is running.

Numbers are entered into a program using the instruction cin. This works the same way as cout, except that it provides the numbers rather than accepting them:

```
// This program accepts two numbers and adds them together
#include <iostream.h>
void main ()
{ int first_num, second_num, result;
  cout << "Please enter two numbers to add together."
    << endl
    << "Please enter the first number ";

  cin >> first_num;

  cout << "Please enter the second number ";

  cin >> second_num;

  result = first_num + second_num;
  cout << endl << "The sum of the numbers is "
    << result;
}
```

This is the only thing that I have shown which approximates to a useful program. Right, we had better go through this bit by bit.

The new bits are the lines in light blue. The instruction cin waits until the user types something and presses Enter. Whatever has been typed is passed to either first_num or second_num depending on which statement it is. This had better be a whole number - those variables are both integers, and if it is anything that can't be stored as an int, the program is going to complain and crash!

You will notice that >> passes items from left to right, just as << passes it from right to left.

Common sense indicates that you should be able to do input like this:

```
first_num << cin;
```

Yes, I thought so too, but I tried it and C++ doesn't like it. Well, what can I say - it's a stupid language!

Run Time Errors

The program listed above only works properly if you type whole numbers when you are prompted. If you type anything else (such as a decimal or a string) then the program will fail. Exactly what the

program will do depends on which version of C++ you are using and exactly what you entered. It might:

- Crash with some error message (the most likely option)
- Work out the sum but based upon some crazy numbers giving a stupid answer
- Try to make sense of the number you provided (e.g. it might treat the decimal 3.4 as 3 and 4 giving the answer 7)

These errors are called *Run Time Errors* as they only happen when you run the program. You can't blame the compiler for not picking them up as it didn't know what you were going to type when it was translating the program. Errors which the compiler does identify are called *Compile Time Errors*.

Here's another example. I have changed the program above so that it divides the first number by the second:

```
// This program accepts two numbers and adds them together
#include <iostream.h>
void main ()
{ int first_num, second_num;
  int result, remainder;
  cout << "Please enter two numbers to divide."
    << endl
    << "Please enter the first number ";

  cin >> first_num;

  cout << "Please enter the second number ";

  cin >> second_num;

  result = first_num / second_num;
  remainder = first_num % second_num;
  cout << endl << "The quotient of the numbers is "
    << result << endl << endl
    << "The remainder of the division is "
    << remainder;
}
```

You'll notice a few improvements. I have put in a calculation to give the remainder as well as the quotient. If you enter numbers such as 19 and 7 when prompted, then all goes well:

```
Please enter two numbers to divide.
Please enter the first number 19
Please enter the second number 7
The quotient of the numbers is 2
The remainder of the numbers is 5
```

The problem occurs when you enter 0 as the second number. If you divide any number by 0, any computer or calculator will give an error. In the C++ system that I use (Borland C++ version 4.5), the following error message pops up:

Char Variables

So far, the variables that we have been dealing with have been numbers, but what if you want to store characters and strings as variables? Storing characters is easy. There is a data type called char which can hold single characters:

```
#include <iostream.h>
void main ()
{ char my_initial;
  cout << "What is your middle initial? ";
  cin >> my_initial;
  cout << "You typed " << my_initial << endl;
}
```

Things are a little harder if you want to store a string. Standard C++ has no 'string' type, although there may be one in the version of C++ that you are using. Strings are stored as arrays of characters - see a later chapter.

The char data type is intended to store characters. However, it stores these characters as numbers in the range -127 to 128. For instance, the letter A is stored as 65, B as 66, lower case 'a' as 97, space as 32 etc.

This means that char can be used simply to store numbers just as int would be, as long as you restrict yourself to numbers between -127 and 128 inclusive. You can use char variables in calculations willy-nilly, but be prepared for the fact that if you try to display a char variable, you will get the character rather than the number!

```
#include <iostream.h>
void main ()
{ char a,b;
  int x;
  // The following line sets 'a' to character M
  a = 77;
  b = a + 1; // 'b' becomes 78
  cout << b << endl; // Displays N, not 78
  x = b;
  cout << x << endl; // Displays 78, as x isn't char
}
```

In the program above, the char variable b is displayed as N, even though it got its value through arithmetic. However, x is an int variable, so it is displayed as a number.

More on Comments

So far you have met comments that start with //. The compiler ignores anything that comes after this up to the end of the line. I have put a comment line at the start of most programs, but you can put them on lines with program statements if you want:

```
#include <iostream.h> // To enable input & output
void main ()          // This is the start
{ char my_initial;    // Declaration of variable
  cout << "What is your middle initial? ";
  cin >> my_initial; // Get user to type initial
  cout << "You typed " << my_initial << endl;
}                    // This marks the end of the program
```

There is another way of doing comments, which is to put them between symbols like this /* and */. In this case, you can put program code after the comment on the same line, because the comment

ends with `*/`. Here I have adapted the program above in a very silly way, putting comments in the middle of the code:

```
#include <iostream.h>
void main ()
{ char /* Hello! */ my_initial;
  cout << "What is your middle initial? ";
  cin >> /* It's me again! */ my_initial ;
  cout << "You typed " << my_initial << endl
  /* Goodbye! */;
}
```

Conditions

Here is an example of a short program with a condition in it:

// Your first conditional program?

```
#include <iostream.h>
void main ()
{ int x;
  cout << "Please type a whole number from 1 to 10 : ";
  cin >> x;
  if (x == 5)
    cout << "You typed my favourite number!" << endl;
    cout << "Thank you and goodbye!" << endl;
}
```

This program asks you to type a whole number from 1 to 10. You do so, and it then tests it to see if it is equal to 5 (the `(x == 5)` part), and if it is, it prints out the message about 5 being its favourite number (Ahh!)

The `(x == 5)` is called the condition. You can see that it is either true or false, depending on what you typed for the number. The brackets are essential - C++ complains if you don't put them in.

Note that you don't need to put `;` after the `(x == 5)`. This is because `(x == 5)` is not the end of the statement. That only happens when the instruction after the condition has been carried out. The `cout` instruction definitely does need `;` after it.

Note also the double equal sign `==` in the condition. This is necessary in a condition to see if two things are equal to each other. We used a single equals sign in variable assignment, but C++ treats testing and assignment as different:

`a = 2*my_age` Setting the value of a

`(a == 2*my_age)` Testing the value of a

If you want to see what a difference an extra equals sign makes, try replacing `(x == 5)` with `(x = 5)`. Whatever number you type now, the condition always passes. The reason is that `(x = 5)` is *making* x equal to 5. The condition then returns the number 5, which C++ interprets as TRUE, so whatever you type, the condition is always true.

`==` isn't the only test you can do. Here are some others:

<	Less than	This returns true if the expression on the left is less than the expression on the right, so <code>a+b < 2*first_try</code> returns true if <code>a+b</code> is less than <code>2*first_try</code> .
>	Greater than	This returns true if the expression on the left is bigger than the expression on the right, so <code>6.1*(a+b) > 2*first_try</code> returns true if <code>6.1*(a+b)</code> is greater than <code>2*first_try</code> .
<=	Less than or equal to	This returns true if the expression on the left is less than or the same as the expression on the right, so <code>a+b <= 2*first_try</code> returns true if <code>a+b</code> works out at less

		than 2*first_try or if they both produce the same number. If a+b comes to greater than 2*first_try, then it produces false.
>=	Greater than or equal to	This returns true if the expression on the left evaluates to more than or the same as the expression on the right, so 30 >= x*y returns true if 30 works out at greater than x*y or if they both produce the same number. If 30 comes to less than x*y, then it produces false.
!=	Not equal to	This returns true if the expression on the left comes to something different from the expression on the right, so a+b != 2*first_try returns true if a+b works out at a different value to 2*first_try. If a+b comes to the same as 2*first_try, then it produces false.

Blocks of Statements

An IF statement often needs to control several statements at once. You group statements together into blocks using the { and } symbols. This example shows an IF statement that prints out several lines of text:

```
if (x == 5)
{ cout << "Well done!" << endl;
  cout << "You guessed my number." << endl;
  cout << "What's your favourite number?" << endl;
}
```

Here you have several statements, each ending with ; and surrounded by { and }. If the condition passes, all three statements are carried out. If the condition fails, none of them is carried out.

The ELSE statement

If you compare the IF statements that we have been using to the one that I gave you at the top of this section you will notice a bit missing - the ELSE part.

The ELSE part of the statement is optional - we haven't needed it up to now. It includes one or more statements (blocked together using { and } in the case of more than one) that are carried out if the condition *fails*.

```
cout << "Please type the number 8 : ";
cin >> x;
if (x == 8)
  cout << "Thank you! I appreciate that." << endl;
else
{ cout << "No, blockhead!" << endl;
  cout << "Why can't you follow simple instructions?"
    << endl;
}
```

Here there is only one statement in the main part of the IF statement, so it doesn't need to have { and } round it. The else part contains two statements, so it definitely does need { and }.

There is a ; after the first cout statement. It looks a little odd coming before the else part, but it is just following a statement, so you should be used to it by now.

++ and --

You would put the symbol ++ after a variable name in order to add 1 on to it. For instance, **my_try++** means increase the value of the variable 'my_try' by 1. This is the instruction that gives C++ its name - it is one step up from C.

```
#include <iostream.h>
void main ()
{ int x;
  x = 12;
  cout << "The value of x is " << x << endl;
  x++;
  cout << "The value of x is now " << x << endl;
}
```

The value of x is 12

The value of x is now 13

You won't be surprised to hear that -- subtracts 1 from the variable:

```
#include <iostream.h>
void main ()
{ int ddx;
  ddx = 37;
  cout << "The value of ddx is " << ddx << endl;
  ddx--;
  cout << "The value of ddx is now " << ddx << endl;
}
```

The value of ddx is 37

The value of ddx is now 36

There is another version of ++ and -- with a slightly different meaning. In this case, the symbols are placed *before* the name of the variable. This still adds/subtracts 1 to/from the variable. The exact difference is explained in a later section (Be patient!)

```
#include <iostream.h>
void main ()
{ int up, down;
  up = 100;
  down = 100;
  cout << "The value of up is " << up << endl;
  cout << "The value of down is " << down << endl;
  ++up;
  --down;
  cout << "The value of up is now " << up << endl;
  cout << "The value of down is now " << down << endl;
}
```

The value of up is 100

The value of down is 100

The value of up is now 101

The value of down is now 99

The process of increasing a variable is called *incrementing* it, whereas the process of decreasing the variable is called *decrementing* it.

Note also:

X++	<i>Increase the value of x by 1</i>
X+=7	<i>Increase the value of x by 7</i>
X+=1000	<i>Increase the value of x by 1000</i>

Similarly, you can use `--` to decrease the value of a variable by any number you like.

<code>m--</code>	Decrease the value of <code>m</code> by 1
<code>Jjq-=7</code>	Decrease the value of <code>jjq</code> by 7
<code>My_try-=1000</code>	Decrease the value of <code>my_try</code> by 1000

The concept of a loop

A loop is a way of repeating a series of instructions several times. The loop is set up either to repeat a certain number of times or to go round and round until some condition is met. Either way there should be some condition that makes the loop terminate.

Implementing loops

Right, now for the nitty-gritty! Let's start with the FOR loop.

```
for ( <initialisation> ; <terminating condition> ; <increment> )  
    { One or more statements in a block }
```

The three parts of the FOR loop are the bits in brackets. You start by initialising the variable that you are going to use as a counter by setting it to a value, for instance:

```
x = 0;
```

The middle part shows the terminating condition, the thing that makes the loop stop. Let's suppose we want the loop to stop when `x` reaches 200. In this case, we use the following :

```
x < 200;
```

This is because the loop keeps running *all the time that the condition remains true!* As soon as the condition fails, the loop terminates.

The final part is the step that makes the counter variable increase. Normally we will want it to count up (or down) one step at a time, so we will use something like this:

```
x++
```

You don't need a semicolon after this as it is the last thing in the brackets. This pair of brackets is followed by one or more statements which are carried out repeatedly. As with the IF statement, if you want to have more than one statement, you must bracket them with `{` and `}`

Here's the FOR loop in all its glory:

```
// Punishment!  
#include <iostream.h>  
void main ()  
{ int counter;  
  for (counter = 1; counter < 101; counter++)  
    cout << "I must not pick my nose in class."  
    << endl;  
}
```

This writes out the line 100 times. You note that the condition contains the number 101. This is because we want the loop to stop as soon as the value of `counter` goes *above* 100. There isn't a semicolon after the brackets as the `cout` statement forms part of the FOR loop.

Here's another example. This program asks you for a number. Then it adds all the numbers from 1 to that number.

```
#include <iostream.h>  
void main ()  
{ int x, limit, sum;  
  cout << "Please enter a number bigger than 1 : ";
```

```

    cin >> limit;
    sum = 0;
    for (x = 1; x <= limit; x++)
        { cout << "I am adding " << x << endl;
          sum = sum + x;
        }
    cout << endl;
    cout << "The sum of all the numbers from 1 to ";
    cout << limit << " is " << sum;
}

```

AND

Suppose you want to test whether a variable *x* is between 1 and 10. You have to test to make sure it is bigger than 1 and that it is smaller than 10:

```

// Conjunction of conditional clauses
// (whatever that means!)
#include <iostream.h>
void main ()
{ int x;
  cout << "Please type a number between 1 and 10 : ";
  cin >> x;
  if (x > 1 && x < 10)
    cout << "Thank you - you've done well." << endl;
  else
    cout << "No. I said BETWEEN 1 AND 10." << endl;
}

```

The two parts of the condition, *x* > 1 and *x* < 10 are joined together with &&.

The two parts of the condition don't have to refer to the same variable, of course. This example shows an IF statement based on no fewer than three variables:

```

#include <iostream.h>
void main ()
{ int number, x, third;
  cout << "Please enter first number : ";
  cin >> number;
  cout << "Please enter second number : ";
  cin >> x;
  cout << "Enter a number bigger than 6 : ";
  cin >> third;
  if (number == 1 && x == 1 && third > 6)
    cout << "Option AAA" << endl;
  else
    cout << "Option BBB" << endl;
}

```

OR

If you thought the && symbol meaning AND was stupid, just wait until you see the symbol for OR! Here it is:

<condition> || <condition>

No, I didn't believe it first time either, but apparently you put `||` between conditions when you want the whole thing to succeed when either part is true. I think an example is called for ...

```
#include <iostream.h>
void main ()
{ int x;
  cout << "Please type an even number from 1 to 9 : ";
  cin >> x;
  if (x == 2 || x == 4 || x == 6 || x == 8)
    cout << "That is correct!" << endl;
  else
    cout << "Sigh! Wrong as usual!" << endl;
}
```

NOT

The symbol `!` means NOT, so it can be used to switch conditions round to point the opposite way. Here, for example, is a condition which prints a message if `x` is 10:

```
if (x == 10)
  cout << "The value of x is 10" << endl;
```

And here is the same condition the other way round. This time, the message is printed if the value of `x` *isn't* 10:

```
if (!(x == 10))
  cout << "The value of x is 10" << endl;
```

I have put the condition inside *another* pair of brackets, and then put `!` in front of it. This reverses the condition so that it only passes if what is inside the brackets is false. If you miss out the innermost brackets the program will still compile but it will produce different results. Try a statement such as `cout << !x;` to find out what else `!` can do.

Nested Loops

Speaking of loops, did you know that you can stick one loop inside another? No, you probably didn't. Well, it doesn't matter, because you do now.

Here's what I mean. This example program has two loops in it, the DO WHILE loop embedded totally inside the FOR loop:

```
#include <iostream.h>
void main ()
{ int x, answer;
  for (x = 1; x <= 10; x++)
  { cout << "Question " << x << endl;
    cout << "What is the answer to " << 2*x
    << " + " << (30 - x) << " ? : ";
    do
    { cin >> answer;
      if (answer == 2*x + 30 - x)
        cout << "Correct!" << endl;
      else
        cout << "No, try again!" << endl;
    }
    while (answer != 2*x + 30 - x);
  }
}
```

This program tests your arithmetic. The FOR loop goes from 1 to 10, asking ten questions. The questions are all of the form "What is XXX + YYY?" with the two numbers being calculated from the value of x, the FOR loop counter.

SUMMARY OF C++

- C++ programs are written in plain text files which are then translated into machine code ready for the computer to run. The program that translates them is called a *compiler*.
- Programs which intend to include input and/or output (which is all of them) should start with `#include <iostream.h>`.
- The main part of the program should start with `void main ()`. The word `void` means that the program does not return a number to whatever called it (the Operating System, presumably). The word `main` means that this is the main part of the program. The `()` means that there are no parameters passed to the main program.
- The program consists of one or more instructions, called *statements*. The statements that make up the main part of the program are enclosed between `{` and `}`.
- It is possible to block statements together so that they can act as one statement. Statements are blocked using `{` and `}`. Any situation that requires one single statement can have a block of statements instead.
- Each statement ends with a semicolon.
- C++ is very tolerant of the layout of the program. The only place you can't put a new line or a space is in the middle of a keyword or a string constant.
- Things are printed on the screen using `cout` and are passed to `cout` using `<<`
- Comments are included either using `//`, in which case everything else on that line is the comment, or by including the comments between `/*` and `*/`.

Variables

- Data can be stored in a program using variables. These are like small boxes in memory, each variable having a *name*, a *contents* and a *data type*.
- Variables must be given names consisting of letters, digits and/or underscore `_`
- The name must not start with a digit but it can start with a letter or underscore.
- The three common types (there are others) are **int**, **float** and **char**, representing whole numbers (integers), decimal numbers (floating point numbers) and letters (or characters).

```
int x;  
float height;  
char index_letter;  
int a,b,c;  
float mm, age;
```

- Variables must be *declared* before being used. This is done by putting either **int**, **float** or **char** followed the name(s) of one or more variables. If you are going to declare more than one variable in one line, then separate them with commas. Either way, the line should end with a semicolon, as any statement should.
- Three other types are **double**, which stores floating point decimal numbers to a higher precision than **float**, **long** which stores large positive or negative whole numbers, and **unsigned int** which is similar to **int** except that it can store higher numbers at the expense of negative whole numbers.
- Variables are assigned to values using `=`.

```
my_try = 3;
```

- Alternatively, variables can be assigned during declaration:

```
float step_value = 3.01;
```

- Or you can get the user to type in a variable value using cin and >>.

Arithmetic

- Arithmetic is done using + (for addition), - (for subtraction), * (for multiplication), / (for division) or % (for the remainder when division is done).
- Be careful when you use / with integer variables - they will produce the whole number part of the division only!
- Multiplication and division are always done before addition and subtraction, unless brackets are put round the addition and subtraction to "promote" it:

```
30 - 6 * 2 produces the value 18
(30 - 6) * 2 produces the value 48
```

- You can put one set of brackets inside another:

```
(2 * (3 + 4) - 7) % 2
```

IF statements

- These test conditions to see whether they are true or not.
- If the condition is true, then one statement or block of statements is carried out.
- If the condition fails, there may be another statement or block of statements that executes (the *ELSE* part).
- Conditions may include any of these operators:

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

- Simple conditions can be joined together to form *compound* conditions using && (for **AND**), || (for **OR**) and ! (for **NOT**).

Loops

- Loops are ways of repeating instructions many times without having to write them out many times.
- Loops can be performed a fixed number of times, or until some condition is met.
- There are three basic types of loop, the FOR loop, the WHILE loop and the DO WHILE loop.

- The the FOR loop has this structure:
- for (<initialisation>; <terminating condition>;
- <increment>)
- { one statement or block of statements }

The the initialisation section tells the program what variable(s) to set up at the start of the loop. The the terminating condition is the condition which determines when the loop stops (the loop runs until the condition *fails*) and the the increment part indicates what the program is supposed to do at the end of each section - normally increasing or decreasing a counter value for the loop.

- The the WHILE loop has this structure:
 - while (<condition>)
 - { one statement or a block of statements }

The loop continues all the time that the condition is true. If the condition fails the first time round, the loop never executes.

SWITCH IN SELECTION

SWITCH is the IF statement de-luxe.

You can test a variable to see if it has any one of several values. Switch statement forws the following syntax:

Here's the syntax:

```
switch (switch_expression)
{
    case constant-expression:
        statement
        jump-statement

    case constant-expressionN:
        statementN
    [default]
}
```

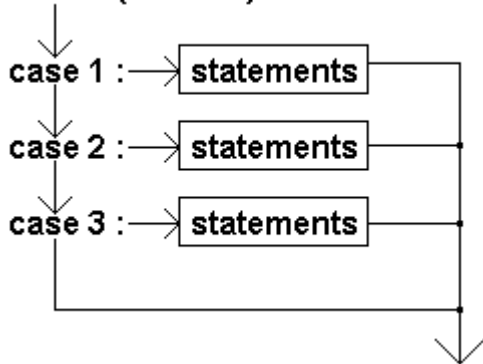
What this means, when translated into English, is ... You follow the word switch with a variable in brackets. Then you put in one or more case clauses. Each word case is followed by a number or an expression which comes out as a constant number, then a colon, and then one or more statements. I'll come to the "break" and "default" bits in a little while.

Perhaps an example with numbers in it might help. Suppose we have a variable called q (Well, I'm getting sick and tired of "x"). q is a score out of 4 in a test. We want to print a message depending on what q is:

```
switch (q)
{ case 0 : cout << "That's a lousy score." << endl;
    break;
  case 1 : cout << "Miserable!" << endl;
    break;
  case 2 : cout << "You could do better, I'm sure!"
    << endl;
    break;
  case 3 : { cout << "Almost there!" << endl;
    cout << "Try harder next time!" << endl;
    }
    break;
  case 4 : { cout << "Well done. A perfect score."
    << endl;
    cout << "Give yourself a pat on the back."
    << endl;
    cout << "You can feel rather smug." << endl;
    }
    break;
}
```

This should be a lot easier to follow. There is no "default" clause at the end. The SWITCH statement has been stripped down to its basics. The variable q is compared to 0, then to 1 etc. and when it matches one of the values given, the appropriate message is printed. The last two options have several statements, and these are bracketed together to form blocks.

switch (variable)



The diagram on the left shows the basic structure of the switch statement.

The reason that the statement break; is included at the end of each case is to tell C++ where each case finishes and the next one starts. You would think that C++ would be able to work it out for itself, but apparently not. To see what I mean, try missing out the break; lines and see what happens.

If you miss out the break statements, you get this effect. Now all the statement blocks run into each other, so whichever case triggered (if any) would cause all the statements from that point on to run.

```
#include <iostream.h>
int main ( )
{
    int num1, num2, op_select;
    float answer;
    cout << "Enter two integers: ";
    cin >> num1 >> num2;
    cout << "Choose an operation: \n"
```

```

cout << "For addition, select 1\n";
cout << "For subtraction, select 2\n";
cout << "For multiplication, select 3\n";
cout << "For division, select 4" << endl;
cin >> op_select;
switch ( op_select )
{
    case 1: answer = num1 + num 2; break;
    case 2: answer = num1 - num2; break;
    case 3: answer = num1 * num2; break;
    case 4: if (num2 == 0 ){
                cout << "Division by zero!"; break;}
            else{
                answer = num1/num2; break;}
    default: cout << "That is not a legal entry" << endl; return 1;
}

```

What is a function?

Functions give a way of splitting a program into manageable chunks. They are routines present in the program which perform a particular task on the data

Using functions gives several advantages. Firstly, they make the program a lot easier to read and understand. The programs that we have been writing so far have pushed the limits of what is easy to read. Any program longer than those that didn't use functions would be hard to understand

Secondly, using functions removes the need to duplicate code. If you had a program which displayed a list of values in an array, then sorted them into alphabetical order, and then redisplayed the values, then the code for displaying the list of values could be written just once, and called twice, once before the sort and once after. This would not be much of a saving since the routine to display a series of numbers is only a couple of lines long, but the same principle can be applied to larger pieces of code.

There are two essential parts to a function, its **declaration** which is the point where it is defined, and one or more **function calls**, which are the points where the program makes use of the function. Here is an example of a (very) simple program which defines a function and calls it twice.

```
#include <iostream.h>
```

```
// This function just adds three numbers
```

```
int add_up (int a, int b, int c)
{ return (a + b + c); }
```

```
// This is the main program
```

```
// It should be familiar from here on!
```

```
void main ()
{ // Here is the first function call
  cout << add_up(3, 4, 12);
```

```
  int x = 12;
```

```

int y = 25;
int z = 140;

// Here is the second function call
float average = add_up(x, y, z-20) / 3.0;
cout << "The average of " << x << " "
    << y << " and " << z << " is " << average;
}

```

Below Is the structure of a fnction

```

<return type> <name of function> (<optional parameter list>)
{
    // The statements making up the function go here

    return <put a return value here>
}

```

Functions must have a name, of course, so that they can be called in the main program

Here is an example of a simple function which returns an **int** value to the calling program and which has no parameters. I chose to call the function **func1** for want of a better name:

```

int func1 ()
{
    cout << "This statement is in function func1";
    cout << endl;
    return 45;
}

```

Here is another function, which I have called **func2**

```

void func2 (int x, float param2)
{
    cout << "This is function func2";
    cout << endl;
    x = 3 + param2;
    values[20][20]++;
}

```

This function does not return a value to the calling program, so its definition starts with the word **void** and there is no **return** statement. It does, however, have two parameters, an integer value which is called **x** within the function and a floating point values which is called **param2** within the function. Parameters are explained later on.

Parameters

It is possible to pass values into functions so that the function can use them. These values are called **parameters**, and, if used, they are placed in a function definition between the brackets just after the name. Here is an example of a function with a single parameter, called **name**:

```

#include <iostream.h>
#include <string.h>

```

```

void greeting (string name)
{
    cout << "Hello, " << name << ", " << endl;
    cout << "Pleased to meet you." << endl;
}

```

```

void main ()
{
    greeting("Richard");
    greeting("Diane");

    string n = "Ludwig van Beethoven";
    greeting(n);
}

```

This program defines the function with one string parameter. The function is called three times. The first time that the function is called, the string constant "Richard" is passed to it, so the parameter **name** becomes "Richard". The second time, the string constant "Diane" is passed, so the parameter **name** becomes "Diane".

Here's an example of a parameter, in this case it is a **double** value.

```

#include <iostream.h>

void another_function (double x)
{
    cout << "In the function x has the value " << x
        << endl;
}

void main ()
{
    double x = 1.23;
    double y = 400.7;

    cout << "In the main program x has the value "
        << x << endl;

    another_function(y);

    cout << "After the function x has the value "
        << x << endl;
}

```

FILE ACCESS IN C++

Writing data to a file

All the functions for file-handling are defined in a header file called `fstream.h` in the same way that the functions like `cin` and `cout` are defined in the header file `iostream.h`, so in order to be able to use any of them, you will need to include the following statement before your code:

```
#include <fstream.h>
```

I suggest you put this just after the `#include <iostream.h>` statement.

The simple way to set up a file so that you can write stuff out to it is like this:

```
const char* fname = "mydata";  
ofstream f(fname);
```

This sets up a string in memory containing the word **mydata** (which will be the name of the file on the disc) and calls that string **fname**. The line **ofstream f(fname);** sets up a file variables called **f** and ties it up with the file specified by **fname** (i.e. "mydata"). The word **const** is an instruction to the compiler telling it that this string is a constant and mustn't be changed anywhere in the program (more on this in the next section).

The word **ofstream** means that the file can only be used for writing data out to the disk - you couldn't use it to read data in from a file.

Opening and Closing Files

Before you can use a file, you have to **open** it. This is done using the name of the file and putting **.open()** after it, like this:

```
const char* name_of_file = "Address2";  
ofstream myfile(name_of_file);  
  
myfile.open(); // This opens the file so that  
               // data can be written to it.
```

This sets up the name of the file as **Address2** and stores it in the constant (unchangeable) string **name_of_file**. The file variable tied up with that string is **myfile**.

There are options which you can put inside the brackets but I won't discuss those here.

Once you have done all the file operations that you need (exactly how you do this is explained in the next section), you should close it. This is done similarly to opening the file except, surprise surprise, you use the word **close** instead of **open**:

```
myfile.close();
```

Writing data out to a file

Right, that's the file opened up for output. How do you actually write data out to it?

Use the file variable (**f** or **myfile** or whatever you put after the **ofstream** instruction) just the same way that you would use `<<` operator that you met at the start of the course.

```
f << "Hello!";  
int x = 10, y = 20, z = x+y;  
f << x << " plus " << y << " gives " << z;
```

After this, the file will contain the following:

```
Hello!10 plus 20 gives 30
```

Whoops! Forgot the new line character:

```
f << "Hello!" << endl;  
int x = 10, y = 20, z = x+y;  
f << X << " plus " << y << " gives " << z;  
Hello!  
10 plus 20 gives 30
```

You can see that the `<<` operator working with files works in exactly the same way as with **cout**. If you miss out things like **endl** (or `\n` - remember that from an earlier section?) then you get an

unexpected result. Your best bet is to write your program using **cout** instead of writing things out to a file. When you have got the layout of the data correct, you can replace the **cout** commands with the appropriate file access commands

An Example

This example program writes the two-times table out to the file on the floppy disc called **TABLE2.TXT**:

```
#include <iostream.h>
#include <fstream.h>

const char* file = "A:\\TABLE2.TXT";
ofstream two_times(file);

void main()
{
    int i;
    two_times.open();

    for (i = 1; i <= 12; i++)
        two_times << i << " times 2 = " << i*2 << endl;

    two_times.close();
}
```

Opening a File for Input

We've dealt with writing data out to a file. What about reading it in? This is done similarly to writing data but with a couple of minor changes.

Firstly, the file has to be declared as an input file rather than as an output file:

```
const char* the_name = "D:\\DATA1.ACC";
ifstream fvar(the_name);
```

```
fvar.open();
```

ifstream works similarly to **ofstream** except that it declares the file to be an "input file stream" rather than an "output file stream". The file is opened using the **open** command in exactly the same way.

You use input files in the same way that you would use **cin**. Again, just like **cin**, they will read up to a space or an end-of-line character, stop, and put what they had read into the variable. For instance, if you had a file containing the following text lines:

**To be or not to be,
That is the question.**

then you might expect the first **f >>** instruction to read the whole of the first line of the quotation into the variable **firstline** and the next **f >>** instruction to read the whole of the second line into **secondline**. Instead, because the file-read instruction stops when it hits a space, **firstline** ends up holding the word **To** and **secondline** holds the word **be**. The string variables will *not* contain the spaces that follow the words in the file (the spaces which caused the **f >>** instructions to stop reading). Indeed, you can separate items in a file by as many spaces or blank lines as you like and the file-read operation will skip them looking for the next item.

Just as with **cin**, if you want to read a string from a file that contains spaces, you should use the function **getline()**. You will recall that the way you use **getline** with **cin** is as follows:

```
cin.getline(destination_variable, maximum_length);
```

This reads a string in from the keyboard (i.e. you type it in), including spaces and all, and only stops when you press Enter or when you have entered the maximum number of characters (specified by the second parameter - you tell it what the maximum number of characters to read is). Input file variables have a similar built-in **getline()** function, so:

```
myfile.getline(s1);
```

In this case, you don't need to specify what the maximum number of characters is. The statement will just read up to the end of the line and then return what it has found.

This reads a whole line from the input file **myfile** into the string variable **s1**. Of course, **getline()** only works for input file variables - there is no **getline()** function for output file variables. (There is a **putline()** function, but that's a different story!)

Detecting the end of a file

Input files (those of type **ifstream**) have a built-in function called **eof()** (standing for **end of file**) which produces a zero answer (i.e. false) if the input file still has data to be read, and a non-zero answer (i.e. true) if the input file has reached the end.

The simplest way to use **eof()** is as follows:

```
if (my_data.eof() == 0)  
    cout << "The file HAS NOT reached the end";  
else  
    cout << "The file HAS reached the end";
```

This is a simple **if** statement. The input file variable is called **my_data** and its **eof()** function is tested to see if it is 0. A more common use of **eof()** is in a loop, which goes round and round and only stops when it reaches the end of the input file:

```
while (!input_file.eof())  
    { \ Loop statements here  
}
```

Note the use of the **!** symbol ('not') meaning "perform this loop while you have *not* reached the end of the input file." You would use a loop like this if you wanted to read through the entire contents of a file, for instance, copying the words to another file or counting the number of times the word "hello" appears etc.

//The notes have been down loaded then modified from the following site:
<http://richardbowles.tripod.com/cpp/intro.htm>. This is for class study ONLY